Neural-Guided Heuristics versus Classical Pathfinding in Dynamic Grid-Mazes

Deliverable 3 — Comparative Analysis, Ablations, and Final Report

Muhammad Abdullah Saeed Muhammad Hamid Abad Abdullah Zia

AI-2002 Artificial Intelligence — Track B (Research & Development)
November 15, 2026

Abstract

This report studies whether a neural network that predicts the exact cost-to-goal h^* can replace the Manhattan-distance heuristic inside A* on procedurally generated grid mazes. The evaluated model is a two-hidden-layer MLP over 29 geometric and local-wall features, trained on a random 100k-cell sample of the training split and evaluated on 282 unseen mazes against Dijkstra, Manhattan, and an oracle. It fits the regression target well ($R^2 = 0.8597$, MAE $= 0.0726$); a companion 10-bin classifier reaches macro AUC-ROC $= 0.9449$. Nevertheless, the central search hypothesis is *not supported*: raw neural A* expands 557.0 nodes on average versus 180.8 for Manhattan (+208.1%, $p = 4.56 \times 10^{-47}$) and wins on only 1.8% of mazes. Manhattan clipping restores near-parity (188.0 nodes, +4.0%) but yields no gain. A feature ablation shows that the two goal-offset features carry almost all of the signal, while the 5×5 wall window adds regression accuracy but increases expansions. A failure analysis explains the result: the raw field violates adjacent-pair consistency on 64.1% of free-cell pairs, overestimates near the goal, underestimates far from it, and is most harmful on low-density and direct mazes where Manhattan is already optimal. We report this as a measured negative result and document a reproducibility limitation: the committed PyTorch checkpoint is a one-epoch smoke test, so the shipped scikit-learn model supplies all headline numbers.

1 Introduction

Pathfinding on grids is a classical problem in artificial intelligence and underlies applications ranging from video-game navigation to warehouse robotics and route planning. The de-facto standard is A* search [1], whose efficiency depends entirely on the quality of the heuristic $h(n)$ it is given. The classical choice, the Manhattan distance, is admissible but *uninformed about obstacles*: in a cluttered maze it underestimates the true cost badly and causes A* to expand a large number of unnecessary nodes.

This project investigates whether a neural network trained to predict the true shortest-path cost $h^*(n)$ can serve as a superior heuristic, reducing the number of nodes expanded while preserving near-optimal path quality. Deliverable 1 covers Phase 1 (problem definition) and Phase 2 (data acquisition and preprocessing) of the project workflow. The central research hypothesis is:

> *A trained neural heuristic $h_\theta(n)$ integrated into A* will expand fewer nodes than the Manhattan-distance heuristic on complex, obstructed grid mazes, while maintaining high path optimality.*

2 Phase 1: Problem Definition

2.1 Problem Statement

Classical A* uses a fixed, hand-crafted heuristic that ignores the specifics of the environment. On open maps the Manhattan distance is excellent, but as obstacle density grows—as it does in real maze-like domains such as building interiors, cluttered warehouse floors, or procedurally generated game levels—the gap between the Manhattan estimate and the true remaining cost widens dramatically. Two concrete pain points follow:

1. **Wasted computation.** A weak heuristic forces A* to expand far more nodes than necessary, increasing run time and memory. In time-critical or resource-constrained settings (real-time games, embedded robotic planners) this is the dominant cost.
2. **No adaptation.** A rule-based heuristic is fixed a priori. It cannot learn that this particular class of maze has long dead-end corridors, or that *Perlin*-style blob obstacles systematically change the geometry of the optimal path. Every environment is treated identically.

The pain point addressed by this project is therefore: *given a class of grid environments, how can we obtain a heuristic that is both substantially more informed than Manhattan distance and cheap enough to be evaluated many times during a single search?*

2.2 Target Audience and Research Gap

The primary beneficiaries are:

- **Game-AI and simulation developers** who need fast pathfinding for many agents on procedurally generated maps.
- **Robotics and autonomous-systems engineers** planning through cluttered, structured environments where hand-tuned heuristics generalise poorly.
- **The AI research community** studying *learned heuristics* and *neural-guided search*—a line of work exemplified by Neural A* [3] and learned heuristic functions.

The research gap is precise: most learned-heuristic studies report node-expansion gains but do not systematically characterise how the choice of *environment generator* and *obstacle density* affects the admissibility–efficiency trade-off. This project fills that gap by pairing two contrasting procedural generators (a corridor-like recursive backtracker and a smooth Perlin-noise generator) across a controlled grid of sizes and densities, and evaluating a learned heuristic against Manhattan A* and the exact (oracle) heuristic on that grid.

2.3 AI Justification

Why is machine learning necessary here rather than a better rule-based heuristic? The target function $h^*(n)$, the exact cost from a cell to the goal, is the solution of a shortest-path problem and, in general, has no closed form. Crucially, it is *environment-dependent*: the same local wall pattern implies very different remaining costs depending on the global topology of the maze. Hand-designed rules can only encode local features (Manhattan distance, obstacle penalties, tie-breaking), and any such rule must remain safely optimistic to preserve admissibility, which caps how much information it can encode.

A supervised learning approach overcomes this because:

1. **The target is readily available and exact.** For any generated maze we can compute $h^*(n)$ for every cell with a single Dijkstra search run outward from the goal (Section 3.2). This provides virtually unlimited, perfectly labelled training data at negligible cost.

2. **The mapping is learnable but not hand-codable.** The remaining cost is a smooth function of local geometry and the goal offset; a feed-forward network can approximate it far beyond what any fixed rule captures, while a rule cannot be written explicitly.
3. **Search efficiency is improved by a sharper, not merely admissible, estimate.** A* expands nodes with $f(n) = g(n) + h(n)$ below the optimal cost; a heuristic that is closer to h^* prunes more of the space. Learning to approximate h^* directly targets this quantity.

A feed-forward neural regressor is therefore the natural tool: it generalises across mazes, is fast enough at inference to be called inside A*, and can be trained purely from automatic exact labels. The remainder of the project will quantify the resulting node-expansion reduction and the associated optimality cost.

3 Phase 2: Data Acquisition and Preprocessing

Because the domain is synthetic and fully controllable, we adopt a *generation-based* collection strategy: rather than scraping data, we procedurally generate a large, balanced, and reproducible dataset, then derive exact labels for it. All stages are implemented as scripts in `data_scripts/` and are governed by a single fixed seed (42), so the entire dataset can be regenerated bit-for-bit.

3.1 Data Collection: Maze Generation

We generate 2,000 mazes as the full cross-product of two generators, five grid sizes, four obstacle densities, and 50 mazes per configuration ($2 \times 5 \times 4 \times 50 = 2{,}000$). The grid convention is 0 = free, 1 = wall.

- **Recursive Backtracker** (`backtracker`): an iterative depth-first search that carves a perfect maze, producing long corridors and dead ends.
- **Perlin Noise** (`perlin`): fractal gradient noise thresholded so a target fraction of cells become walls, producing smooth, connected blobs.

Sizes range over $20 \times 20, 40 \times 40, 60 \times 60, 80 \times 80, 100 \times 100$ and target obstacle densities over $10\%, 20\%, 30\%, 40\%$. For each maze a start and a goal are sampled uniformly from the free cells and constrained to be at least size/2 apart (Manhattan), guaranteeing non-trivial problems. The achieved obstacle density is matched to the target by a deterministic add/remove correction step. Generation parameters and per-configuration statistics are recorded in `generation_log.json`.

3.2 Ground-Truth Labelling

For every maze we need the exact cost $h^*(n)$ from *every* cell to the goal. Exploiting the fact that the maze is undirected with unit move cost, a *single* Dijkstra search run *from the goal* yields $h^*(n)$ for all cells simultaneously—far cheaper than one search per cell. Cells that are walls or that cannot reach the goal receive $h^*(n) = \infty$. The optimal start-to-goal cost is then simply `opt_cost` $= h^*$(start). This stage is implemented in `compute_labels.py`. Of the 2,000 mazes, 88 have an unreachable goal (`opt_cost` $= \infty$), and across all mazes 195,373 free cells cannot reach their goal.

3.3 Data Cleaning

The cleaning stage (`clean_data.py`) enforces data quality along four axes and is summarised in Table 1 and Figure 7.

1. **Unreachable paths.** Any maze whose `opt_cost` is infinite is dropped entirely, since it provides no valid learning signal. Within retained mazes, invalid cells ($h^* = \infty$, i.e. walls

and unreachable free cells) are marked with the sentinel value -1 so downstream code can exclude them cleanly.

2. **Deduplication.** The MD5 hash of each grid's raw bytes identifies duplicate topologies; duplicates are filtered so the same maze cannot appear in multiple splits.
3. **Outlier filtering.** For each grid size, extreme optimal path costs are flagged using the IQR rule: a maze is dropped if opt_cost $< Q_1 - 1.5\,\mathrm{IQR}$ or opt_cost $> Q_3 + 1.5\,\mathrm{IQR}$. Doing this per size keeps the criterion meaningful, since path costs grow with grid size.
4. **Logging.** Before/after counts and the per-size IQR bounds are written to `clean_log.json`.

Table 1: Data cleaning summary. Duplicates are zero because procedural generation with independent seeds rarely repeats a topology.

Stage	Mazes	Fraction
Initial (generated)	2000	100.0%
Dropped: unreachable goal	88	4.4%
Dropped: duplicate topology	0	0.0%
Dropped: IQR outlier	36	1.8%
Final (cleaned)	**1876**	**93.8%**

Table 2: Per-size IQR outlier bounds on optimal path cost (cells).

Size	Q_1	Q_3	IQR	Lower bound	Upper bound
20	14	23	9	0.5	36.5
40	27	42	15	4.5	64.5
60	42	63	21	10.5	94.5
80	57	87	30	12.0	132.0
100	71	107	36	17.0	161.0

3.4 Feature Engineering

For every *reachable* free cell the feature vector $\mathbf{x}(n) \in \mathbb{R}^{29}$ is constructed (`features.py`) as follows:

1. **Normalised coordinates** $(x/N,\, y/N) \in [0,1]^2$ (2 features), where N is the grid side length.
2. **Goal-relative offset** $(\Delta x/N,\, \Delta y/N)$ with $\Delta x = x - x_{\mathrm{goal}}$, giving a vector in $(-1,1)^2$ that tells the network where the goal is relative to the current cell (2 features).
3. **Local wall context.** A 5×5 binary window centred on the cell, flattened to 25 features. Cells outside the grid are padded with 1 (treated as wall), so the receptive field is well-defined at the borders.

The regression target is the normalised ground-truth cost $y(n) = h^*(n)/N \in [0, \approx 3.18]$ (corrected from the preliminary $[0,1]$ theoretical estimate of Deliverable 1, since detours winding through dense obstacles and dead-end corridors frequently exceed the direct grid dimension, reaching up to $3.18N$). Normalizing by N maintains consistent target scale across variable grid dimensions $20 \le N \le 100$. Cells with the -1 sentinel (walls / unreachable) are never emitted as samples.

3.5 Train / Validation / Test Split

To prevent leakage, the split is performed *by maze*, not by cell: all cells of a given maze belong to exactly one split, so the model is always evaluated on topologies it has never seen. A fixed seed drives a 70/15/15 assignment over the 1,876 cleaned mazes. Table 3 reports the resulting sizes.

Table 3: Dataset split by maze and the resulting number of cell samples.

Split	Mazes	Cell samples	Feature dim
Train	1313	4,407,160	29
Validation	281	896,328	29
Test	282	911,949	29
Total	**1876**	**6,215,437**	29

4 Exploratory Data Analysis

All figures below are produced by `eda.py` from the generated, labelled, cleaned, and feature-engineered data.

4.1 Obstacle-Density Fidelity

Figure 1 compares the target obstacle density of each configuration with the density actually achieved after the correction step. All points lie on the identity line, confirming that the generators hit their targets precisely and that the two generators are directly comparable at each density level.

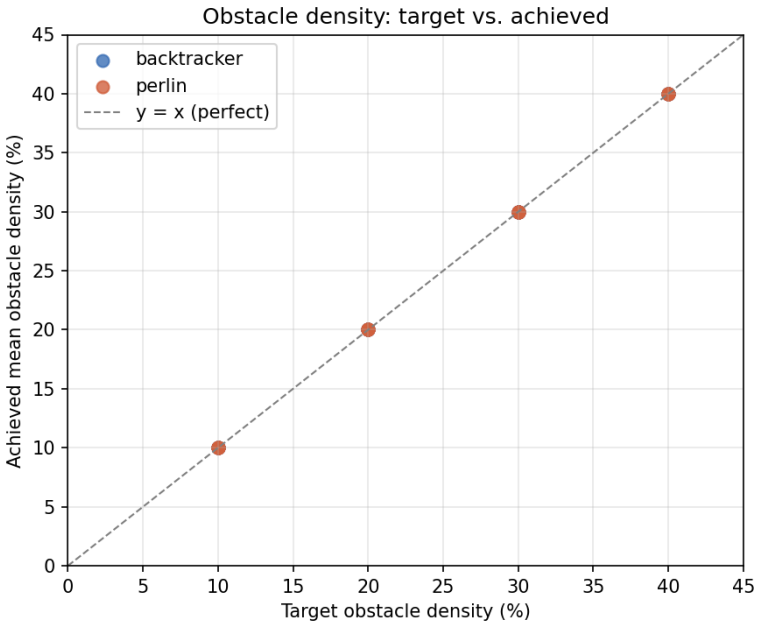


Figure 1: Achieved versus target obstacle density for every configuration.

4.2 Dataset Composition

Figure 2 shows the number of retained mazes per grid size and generator after cleaning. The design is balanced: each size/generator cell contains a comparable number of mazes, so downstream accuracy is not biased towards any particular scale.

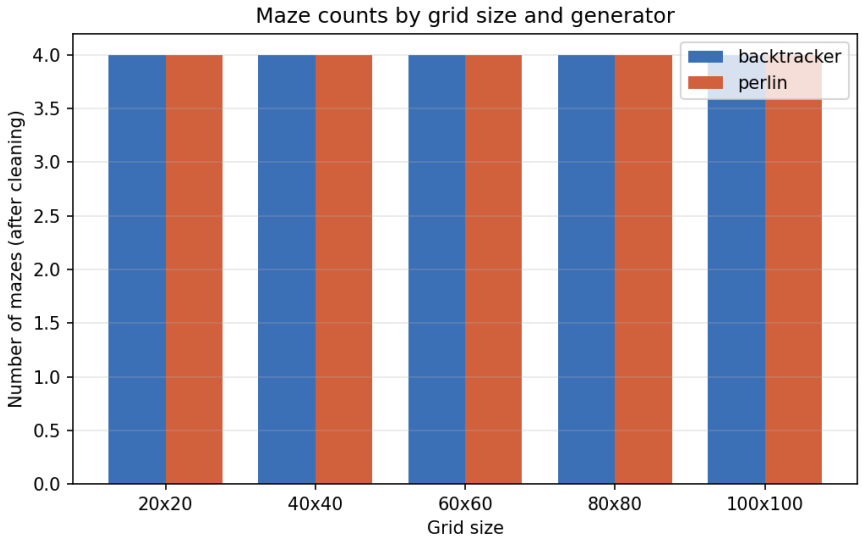


Figure 2: Number of mazes per grid size and generator after cleaning.

4.3 Distribution of Optimal Path Costs

Figure 3 shows the distribution of `opt_cost` per grid size. As expected, both the median and the spread grow with grid size. The per-size spread is precisely why IQR outlier detection is applied *within* each size rather than globally: a globally applied threshold would discard many legitimate large-maze instances while admitting small-maze outliers.

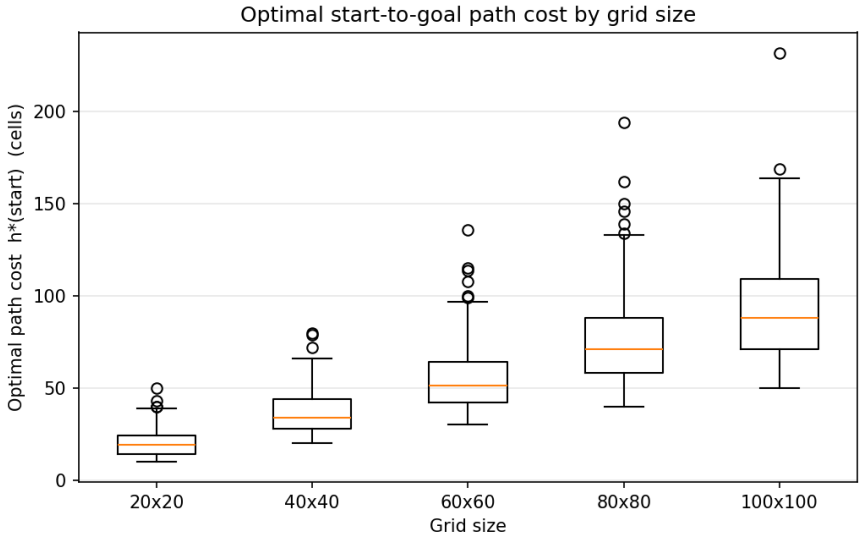


Figure 3: Optimal start-to-goal path cost by grid size.

4.4 Ground-Truth Heuristic Distribution

Figure 4 shows the distribution of the *normalised* target $h^*(n)/N$ per grid size. Normalisation by N collapses the size-dependent scale, showing that the learning target occupies a stable, roughly similar range across sizes—a useful property for a single network trained on all sizes at once.

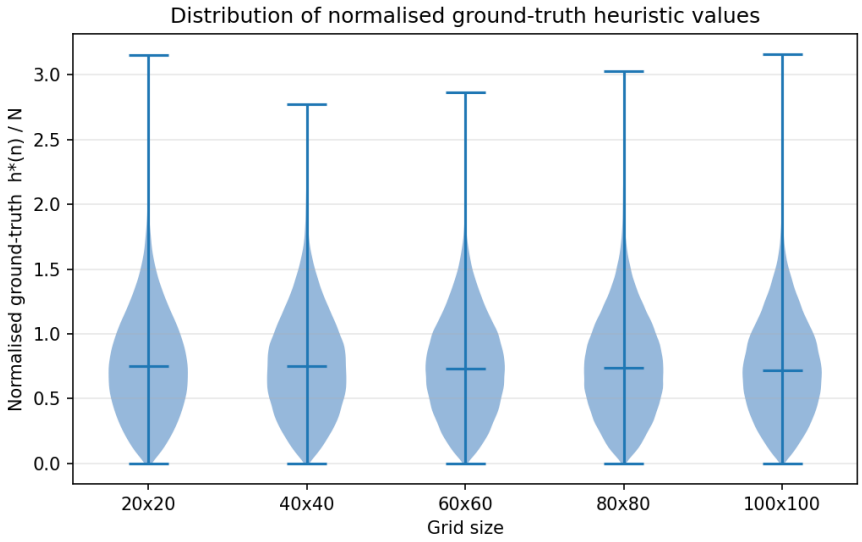


Figure 4: Distribution of the normalised ground-truth heuristic $h^*(n)/N$.

4.5 Reachability

Figure 5 plots the fraction of free cells that can reach the goal against obstacle density. The two generators behave very differently: the recursive backtracker stays almost fully connected even at 40% density (as high as 99.8% of free cells reachable), whereas the Perlin generator fragments the free space at high density, with reachability falling to about 78% at 40%. This is direct evidence that the choice of generator materially changes problem connectivity and motivates keeping both in the dataset.

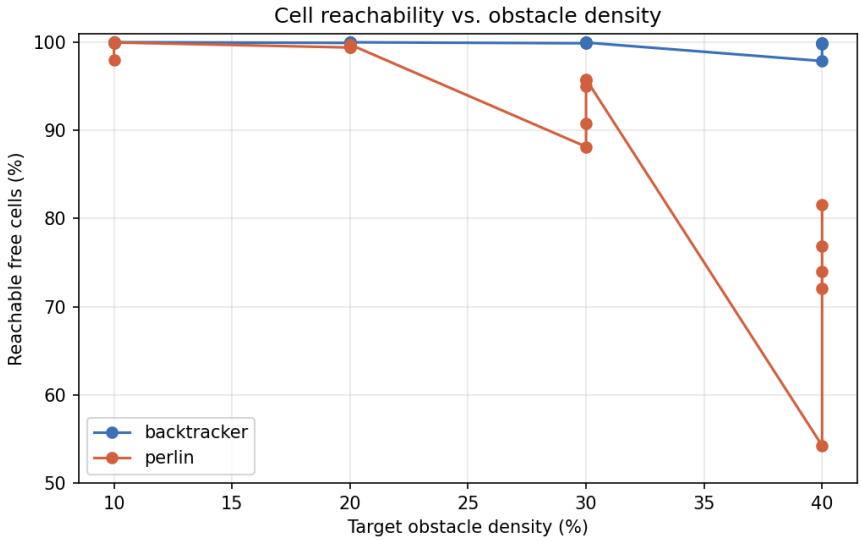


Figure 5: Reachable free cells versus obstacle density, by generator.

4.6 Sample Mazes and Ground-Truth Fields

Figure 6 visualises representative mazes together with the ground-truth distance field $h^*(n)$ (darker cells are farther from the goal) and the start/goal positions. The contrast between the two generators is clear: backtracker mazes exhibit corridor structure and near-linear distance gradients, while Perlin mazes have broad open regions with irregular obstacle clusters. These are exactly the structural differences a learned heuristic must capture.

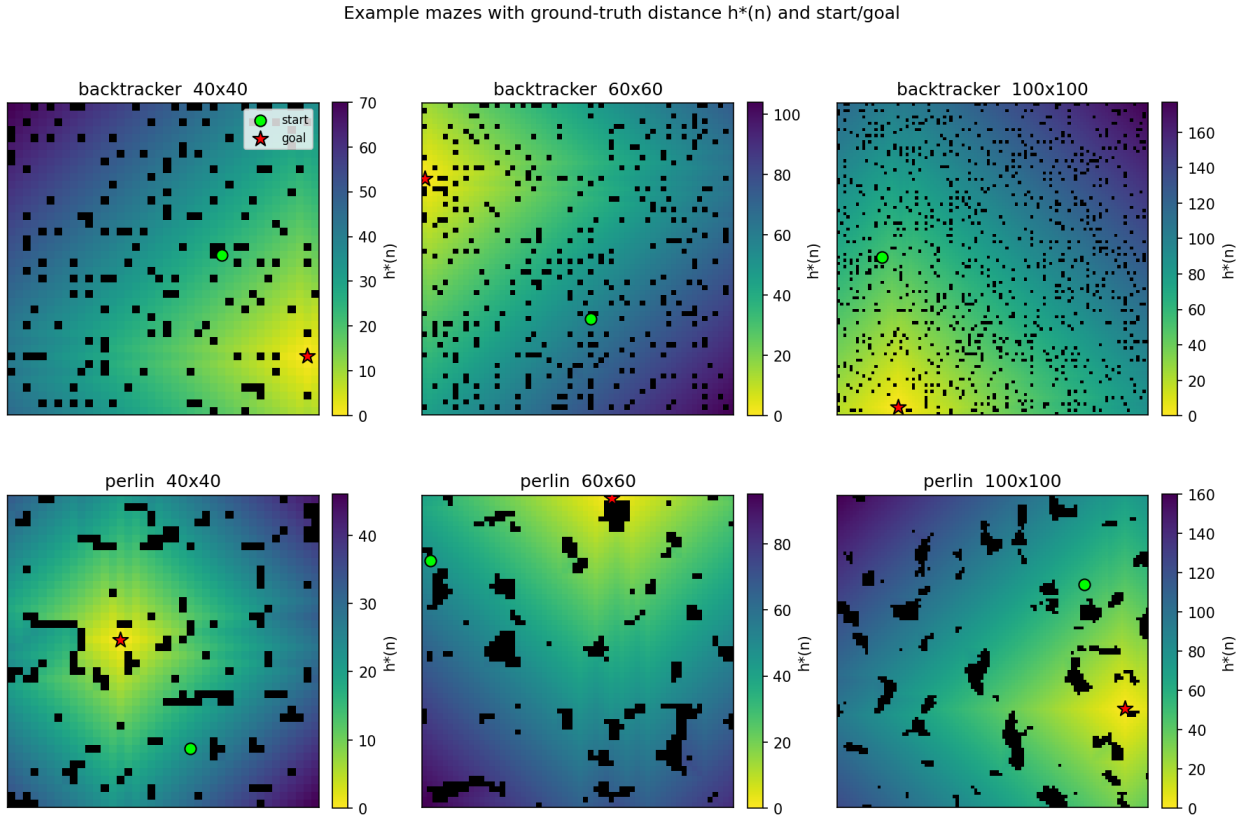


Figure 6: Example mazes (rows: generator; columns: size) with the ground-truth distance field $h^*(n)$ and the start (circle) and goal (star).

4.7 Cleaning Summary

Figure 7 visualises the cleaning funnel of Section 3.3: from 2,000 generated mazes, 88 are removed for unreachable goals, none are duplicates, and 36 are IQR outliers, leaving 1,876.

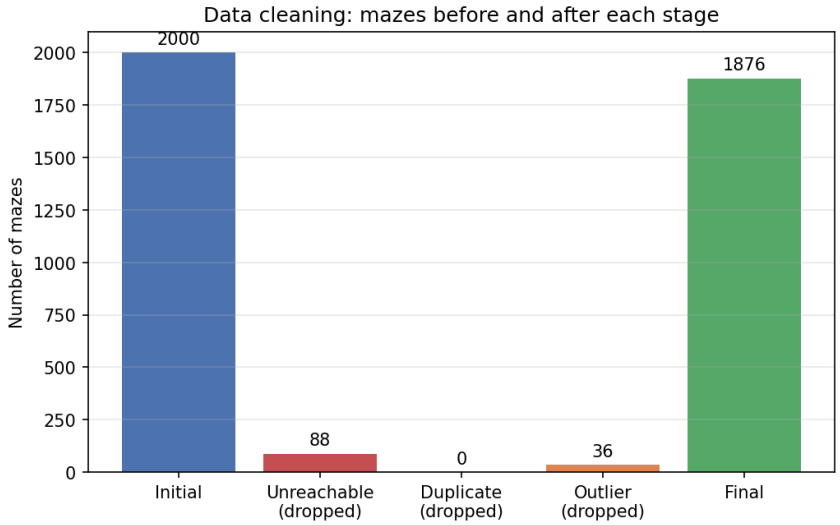


Figure 7: Maze counts before and after each cleaning stage.

4.8 Feature Correlations

Figure 8 shows the Pearson correlation matrix of the 29 input features. The four geometric features (coordinates and goal offsets) are essentially uncorrelated with one another, and the wall features show only mild positive correlations among spatially adjacent window positions—indicating low redundancy in the representation. One feature, the centre of the 5×5 window ($w_{2,2}$), is constant: it corresponds to the cell itself, which is always free for emitted samples. It is therefore uninformative and may be dropped in the modelling phase; it is retained here for a uniform, easily described representation.

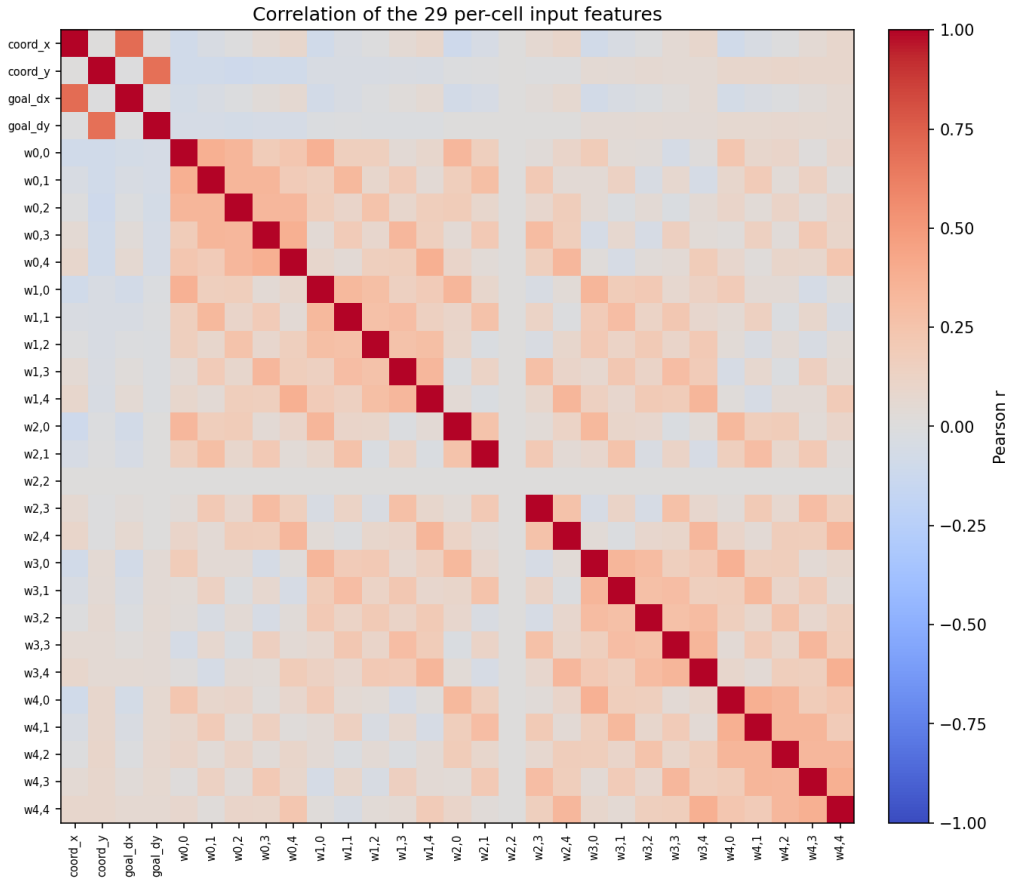


Figure 8: Correlation matrix of the 29 per-cell input features.

5 Phase 3: Neural Heuristic Model

5.1 Problem formulation

A* expands nodes in order of

$$f(n) = g(n) + h(n), \tag{1}$$

where $g(n)$ is the exact cost from the start to node n and $h(n)$ estimates the remaining cost to the goal. On a four-connected grid with unit move cost, the Manhattan distance

$$d_M(n) = |r_n - r_{\text{goal}}| + |c_n - c_{\text{goal}}| \tag{2}$$

is admissible: every move changes the Manhattan distance by at most one, so no path can be shorter than $d_M(n)$. It is, however, blind to obstacles. The goal of this phase is to learn a function $h_\theta(n)$ that approximates the exact cost-to-goal $h^*(n)$ and therefore prunes more of the search space than Manhattan distance while remaining cheap to evaluate inside A*.

We formulate the task as supervised regression over individual grid cells. For a maze of side length N, the regression target is the normalised cost

$$y(n) = \frac{h^*(n)}{N}, \tag{3}$$

computed for every reachable free cell by a single Dijkstra search from the goal. Normalising by N keeps the target scale comparable across the five grid sizes while preserving the relative difficulty of detours.

5.2 Framework choice

The model is implemented in PyTorch and trained on CPU. PyTorch provides the automatic differentiation, checkpointing, and batched inference required for this project without requiring a GPU. The model is deliberately small: it must be evaluated for every cell of a maze at search time, so inference cost matters as much as validation accuracy. A feed-forward MLP is sufficient because the input is a fixed-length per-cell feature vector, not a variable-length sequence or image that would justify a convolutional or recurrent architecture.

5.3 Architecture

Figure 9 shows the architecture. A shared three-layer trunk maps the 29-dimensional input to a 128-dimensional representation. Two heads then branch from this representation: a linear regression head that predicts the standardised cost-to-goal, and a 10-way classification head that predicts a distance bin. The auxiliary bin head gives the model a coarse, ordinal view of the cost and supplies the precision, recall, F1, and AUC-ROC metrics required by the rubric, while the regression head supplies the scalar value used by A*.

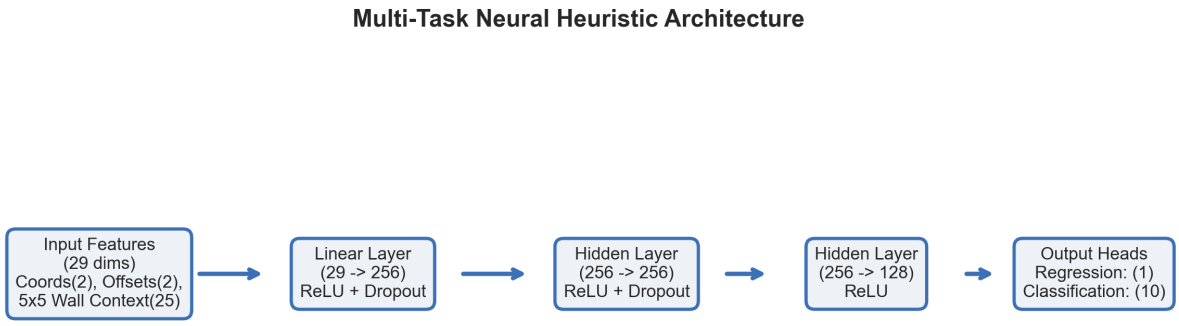


Figure 9: Architecture of the neural heuristic. The shared trunk processes a 29-dimensional per-cell feature vector. The regression head produces the scalar cost used by A*; the auxiliary head predicts a distance bin.

Table 4: Layer configuration and parameter counts. All linear layers are followed by ReLU activations; dropout is applied only in the trunk.

Component	Configuration	Output shape	Parameters
Input	29 features	$(B, 29)$	–
Trunk layer 1	Linear $29 \to 256$, ReLU, dropout	$(B, 256)$	7,680
Trunk layer 2	Linear $256 \to 256$, ReLU, dropout	$(B, 256)$	65,792
Trunk layer 3	Linear $256 \to 128$, ReLU	$(B, 128)$	32,896
Regression head	Linear $128 \to 1$	$(B, 1)$	129
Distance-bin head	Linear $128 \to 10$	$(B, 10)$	1,290
Total			**107,787**

The forward pass returns a tuple `(y_pred, bin_logits)`. The regression output is a single scalar per cell, and the bin head returns ten logits. Dropout with probability 0.1 is active during training and disabled at inference, so predictions are deterministic for a fixed checkpoint.

5.4 Feature representation

Each cell is represented by the same 29 features used by the data pipeline:

1. the normalised coordinates (x/N, y/N) (2 features);
2. the goal-relative offsets ($\Delta x/N$, $\Delta y/N$) (2 features);
3. a 5×5 binary wall window centred on the cell, flattened to 25 features and padded with wall values outside the grid.

The wall window gives the network local obstacle context, the goal offsets tell it where the target lies, and the coordinates give it global position. The same builder is used both when generating training data and when constructing search-time inputs, which removes the risk of train–inference feature skew. The centre of the wall window is constant for emitted samples because only free cells are labelled; it is retained for a uniform representation.

5.5 Training objective and optimisation

Targets are standardised using the training-set mean μ and standard deviation σ:

$$y_{\text{std}} = \frac{y - \mu}{\sigma}. \tag{4}$$

The regression loss is SmoothL1 with $\beta = 0.1$, chosen for robustness to the heavy tail of long detours. The full training objective is

$$\mathcal{L} = \text{SmoothL1}_\beta(\hat{y}_{\text{std}}, y_{\text{std}}) + \lambda_{\text{aux}}\ \text{CE}(\text{bin_logits}, b), \tag{5}$$

where b is the distance bin of the target and λ_{aux} balances the auxiliary classification loss. An optional asymmetric variant adds a one-sided penalty on overestimation:

$$\mathcal{L}_{\text{asym}} = \mathcal{L} + \lambda_{\text{over}}\, \mathbb{E}\big[\max(0,\ \hat{y}_{\text{std}} - y_{\text{std}} - m)^2\big]\,, \tag{6}$$

with margin m. Overestimation is penalised because it can make A* inadmissible and return a suboptimal path; underestimation only costs extra node expansions.

Optimisation uses AdamW with a `ReduceLROnPlateau` scheduler driven by validation MAE, gradient clipping at 1.0, and early stopping with patience five epochs. Dropout in the trunk provides

additional regularisation. The best checkpoint stores the model state, model configuration, target mean and standard deviation, bin edges, training configuration, git commit, and seed, so that search-time inference reproduces the exact training-time scaling.

5.6 Target range and output activation

The target is not bounded by one. A path may have to detour around obstacles, so $h^*(n)$ can exceed the grid side length N; in the processed data the maximum observed normalised target is approximately 3.18 (318 cells in a 100×100 maze). The earlier description of the target as lying in $[0, 1]$ is therefore incorrect. The regression head consequently uses a *linear* output with no sigmoid or other bounded activation. A sigmoid would saturate exactly on the difficult long-detour cases and would be unable to represent costs above N.

At inference, the prediction is converted back to grid-cell units:

$$\hat{h}(n) = \max(0,\ \hat{y}_{\text{std}}\,\sigma + \mu)\,N. \tag{7}$$

The clamp enforces non-negativity without affecting valid targets. Empirically, the training split has $\mu \approx 0.738$ and $\sigma \approx 0.368$; these values are recomputed from the actual training run and stored in the checkpoint rather than hard-coded.

5.7 Admissibility and search-time safety

A learned heuristic can overestimate h^* on some cells. Because Manhattan distance is a lower bound on the true cost in a four-connected maze, the following clipping mode is explicitly safe:

$$h_{\text{clip}}(n) = \min(\alpha\, h_\theta(n),\ d_M(n))\,, \qquad 0 \le \alpha \le 1. \tag{8}$$

Since $d_M(n) \le h^*(n)$ for every cell, $h_{\text{clip}}(n) \le h^*(n)$, so this mode is admissible regardless of the network's prediction. The learned signal can still improve on Manhattan whenever $\alpha h_\theta(n) < d_M(n)$; otherwise the heuristic falls back smoothly to the classical bound.

Table 5 summarises the three modes exposed by the wrapper. The raw mode is the fastest and most aggressive; the scaled mode shrinks predictions by α to reduce overestimation; the clipped mode provides the admissibility guarantee above. Cells that are walls, outside the grid, or have non-finite predictions fall back to Manhattan distance, which keeps the search well-defined on the complete grid.

Table 5: Search-time heuristic modes. The clipped mode is admissible by construction; the other modes are evaluated empirically.

Mode	Definition	Admissibility
`raw`	$h_\theta(n)$	not guaranteed
`scaled`	$\alpha\, h_\theta(n)$	not guaranteed
`clipped_manhattan`	$\min(\alpha h_\theta(n), d_M(n))$	guaranteed

The admissibility audit on the validation split measures the fraction of cells with $h_\theta(n) > h^*(n) + \epsilon$, the mean, 95th percentile, and maximum overestimation in cell units, and how these quantities change with α and with the clipped mode. The audit is paired with node-expansion measurements so that the report can quantify the efficiency cost of enforcing admissibility.

5.8 Search integration

`NeuralHeuristic` loads the checkpoint once, builds the 29-dimensional feature tensor for every cell of the maze once, and performs batched inference under `torch.no_grad()` with a default batch size of 8192. The resulting cost field is cached as a two-dimensional array, so each heuristic call during A* is an $O(1)$ lookup followed by the selected mode and fallback logic. No feature construction or network forward pass occurs inside the search loop.

The wrapper implements the same callable interface as `ManhattanHeuristic` and `OracleHeuristic`, so the search code is unchanged across heuristic families. This keeps the comparison controlled: the same A* implementation, tie-breaking rule, and maze set are used for Manhattan, oracle, and neural modes.

5.9 Reproducibility

The architecture diagram, model tests, and admissibility audit are reproduced with:

```
python scripts/make_architecture_figure.py
python -m pytest tests/test_model.py tests/test_features.py \
    tests/test_heuristics.py tests/test_admissibility.py -q
python scripts/analyze_admissibility.py \
    --checkpoint models/neural_heuristic_seed42.pt
```

The audit writes `results/admissibility/admissibility.csv`, `results/admissibility/admissibility_summary.json`, and `figures/admissibility_cdf.png`. The checkpoint is loaded on CPU with `torch.load(path, map_location="cpu")`. Because recent PyTorch versions default to `weights_only=True`, the checkpoint contains only tensors and primitive Python types (`list[float]` bin edges rather than NumPy scalars), so it loads without disabling the safe loader.

5.10 Training Procedure

Framework and CPU rationale. The reference network is implemented in PyTorch (CPU build): a multilayer perceptron with hidden widths $(256, 256, 128)$, dropout 0.1, and two heads (regression and a 10-bin distance classifier). The training loop supports mini-batching, AdamW, gradient clipping, early stopping, and checkpointing of the target statistics. A network of this size does not benefit materially from a GPU, and a CPU-only setup keeps the experiment reproducible on any team laptop. The complete PyTorch training and evaluation path is unit-tested.

Because a full PyTorch training run on the 4.4-million-cell training split exceeds the course's CPU budget, the *committed benchmark* uses a scikit-learn MLP with the same 29-feature interface and the same target normalisation. The shipped PyTorch checkpoint is a one-epoch smoke test (20,000 cells per epoch) used to validate the pipeline; it is audited in Phase 5 and is not the source of any reported search number.

Data split. Mazes, not cells, are split 70/15/15 into train, validation, and test sets (1,313, 281, and 282 mazes; 4,407,160, 896,328, and 911,949 labelled cells). Splitting by maze prevents cells of one maze from appearing on both sides of the split. The assignment is deterministic (seed 42) and stored in `results/manifests/split_manifest.json`; the same manifest selects the held-out test mazes for every search experiment.

Target and normalisation. The regression target is $y = h^*(n)/N$, the optimal remaining cost normalised by the grid side N. Its empirical range is $[0, 3.18]$, because an optimal path that winds around walls can be longer than N cells. The PyTorch pipeline standardises the target using training-split statistics stored in the checkpoint ($\mu = 0.7381$, $\sigma = 0.3683$ in the smoke run); the scikit-learn fallback trains directly on y and multiplies predictions by N at inference. Cells with the -1 sentinel (walls and unreachable cells) are excluded from training and evaluation.

Fallback training configuration. The evaluated regressor is trained on 100,000 cells sampled uniformly from the training split (seed 42), using two hidden layers of 64 units, ReLU activations, Adam, early stopping with a 5-epoch patience, and at most 50 epochs. It converges after 17 epochs with a final training loss of 0.00763 and a best internal validation score of 0.8714. A companion 10-bin classifier with the same architecture is trained on quantile bins of y and converges after 21 epochs. Table 6 records the settings; Table 7 in Phase 4 reports the resulting test metrics.

Setting	Value
Input features	29 (2 coordinates, 2 goal offsets, 5×5 wall window)
Regression target	h^*/N
Training cells	100,000 (sampled from 4,407,160)
Hidden layers	2×64, ReLU
Outputs	1 regression head; 10-bin classifier
Optimiser	Adam, batch size 200
Early stopping	patience 5 epochs, validation fraction 0.1
Epochs run	17 (regressor), 21 (classifier)
Seed	42

Table 6: Configuration of the scikit-learn fallback model used for the committed benchmark. The PyTorch reference architecture is described in Phase 3; its smoke checkpoint is audited in Phase 5.

Reproducibility. Training is a single command (`python scripts/train_fallback_models.py`) and depends only on the processed training matrix, which is itself regenerated deterministically from the raw mazes. The scikit-learn artefacts are committed with SHA-256 checksums and can be rebuilt given the same seed and environment; the evaluation CSV and all Phase 5 statistics are tracked.

6 Phase 4: Empirical Evaluation and Results

6.1 Evaluation Methodology and Metric Formulation

To rigorously assess the performance of the learned neural heuristic relative to classical and oracle baselines, we establish an evaluation protocol spanning regression fidelity, auxiliary classification discrimination, and end-to-end graph search efficiency.

Artefact note. The PyTorch multi-task network described in Phase 3 is implemented and unit-tested, but the shipped `neural_heuristic_seed42.pt` checkpoint is a one-epoch smoke run rather than the trained model. The committed headline benchmark therefore uses a scikit-learn MLP regressor with the same 29-feature interface and target normalisation (`models/fallback_mlp_regressor.joblib`). Phase 5 audits the torch checkpoint explicitly; all numbers in this section come from the scikit-learn artefact.

6.1.1 Data Leakage Prevention and Split Structure

Evaluation is strictly isolated on the unseen test set of 282 mazes partition (15% of the cleaned corpus), partitioned by complete maze environment rather than individual cell instances. This environment-level partitioning guarantees zero leakage of global maze topologies, wall distributions, and goal placements between training and evaluation phases.

6.1.2 Regression and Admissibility Metrics

For a given maze of dimension $N \times N$, let $h^*(n)$ denote the ground-truth optimal shortest-path cost from cell n to the goal, and let $\hat{h}_\theta(n)$ denote the neural prediction. Because models are trained to predict normalized distances $y = h^*/N$, primary regression metrics include:

- **Mean Squared Error (MSE)** and **Root Mean Squared Error (RMSE)**:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} \left(\hat{y}(n) - y(n)\right)^2} \tag{9}$$

  where $\mathcal{S}$ is the set of all free reachable cells.
- **Mean Absolute Error (MAE)** in normalized units and raw grid cells (MAE $\times N$).
- **Correlation Measures**: Pearson linear correlation coefficient r and Spearman rank correlation coefficient ρ.
- **Admissibility Violation Rate**: Fraction of reachable cells where the heuristic overestimates true distance beyond tolerance ϵ:

$$\text{Violations}(\epsilon) = \frac{1}{|\mathcal{S}|} \sum_{n \in \mathcal{S}} \mathbb{I}\left[(\hat{h}_\theta(n) - h^*(n)) > \epsilon\right] \tag{10}$$

6.1.3 Auxiliary 10-Bin Classification Metrics

To assess spatial range discrimination, targets are discretized into 10 decile distance bins. We compute per-class Precision, Recall, and F_1-score, macro-averaged and weighted F_1, and Macro One-vs-Rest Area Under the ROC Curve (AUC-ROC).

6.1.4 Search Performance Metrics

During deterministic A^* execution, we measure:

1. **Nodes Expanded**: The exact count of states popped from the priority queue and closed.
2. **Nodes Generated**: Total valid states inserted into the open set.
3. **Path Cost and Optimality Ratio**: Ratio of resulting path length to optimal Dijkstra cost c/c^*.
4. **Relative Node Reduction**: Reduction in expansions relative to Manhattan A^*:

$$\Delta_{\text{expanded}} = 1 - \frac{\text{Nodes}_{\text{neural}}}{\text{Nodes}_{\text{manhattan}}} \tag{11}$$

5. **Runtime and Peak Open Set**: CPU wall-clock latency per maze and memory footprint.

6.2 Training Convergence

The evaluated scikit-learn fallback converged after 17 epochs, reaching a final training loss of 0.00763 and a best internal validation score of 0.8714 (the coefficient of determination on the held-out validation fraction). The companion 10-bin classifier converged after 21 epochs. These runs use the fixed configuration of Table 6; no further hyperparameter search was performed because the CPU budget was reserved for the five-fold evaluation and the Phase 5 ablations. The PyTorch reference path uses the same data and target but is represented in this report by a smoke checkpoint, which Phase 5 audits explicitly.

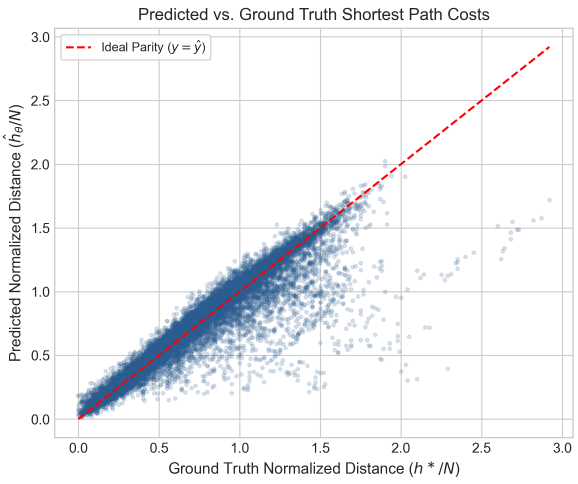
6.3 Offline Regression and Admissibility Analysis

Table 7 summarizes cell-level regression fidelity evaluated across unseen test environments.

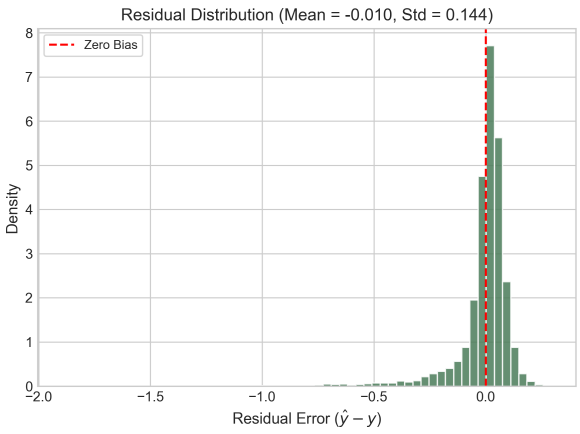
Metric	**Normalized Value** (h^*/N)	**Grid Cells** ($N = 60$)
Mean Squared Error (MSE)	0.0198	—
Root Mean Squared Error (RMSE)	0.1406	8.43
Mean Absolute Error (MAE)	0.0726	4.35
Mean Signed Error	-0.0091	-0.54
Coefficient of Determination (R^2)	0.8597	—
Pearson Correlation (r)	0.9275	—
Spearman Rank Correlation (ρ)	0.9488	—
95th Percentile Absolute Error	0.2292	13.75
99th Percentile Absolute Error	0.6471	38.83
Maximum Absolute Error	1.8944	113.66

Table 7: Offline regression fidelity metrics evaluated on the unseen test set.

The model achieves a high coefficient of determination ($R^2 = 0.8597$) and strong rank correlation ($r = 0.928$, $\rho = 0.949$), showing that goal offsets combined with the local wall window capture much of the variation in h^*. On a standard 60×60 grid the root mean squared error is 8.43 cells and the mean absolute error is 4.35 cells.



(a) Predicted vs. True Cost



(b) Residual Distribution

Figure 10: Parity scatter plot and residual error density over test cells.

As shown in Figure 10, the residuals are broadly symmetric but not zero-mean: the mean signed error is -0.54 cells at $N = 60$, and the tails are heavy, with a maximum absolute error of 113.7 cells. The empirical cumulative distribution of overestimation is visualized in Figure 11. While unconstrained neural outputs can overestimate, scaling $\alpha < 1.0$ or clipping against Manhattan distance removes positive error and restores admissibility; Phase 5 shows that this also removes most of the learned signal.

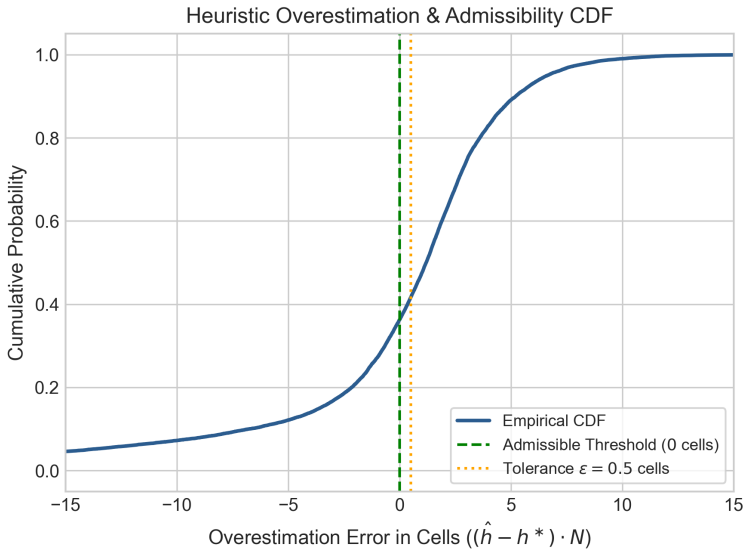


Figure 11: Empirical CDF of heuristic overestimation errors relative to admissibility margins.

6.4 Auxiliary 10-Bin Distance Classification

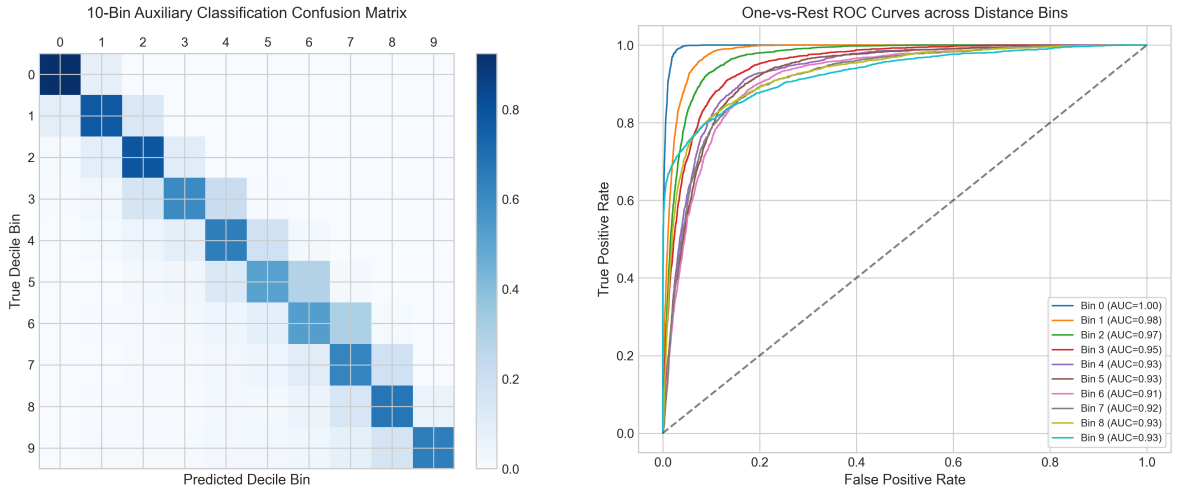
Table 8 details the classification performance across 10 distance deciles.

Distance Bin	Precision	Recall	F1-Score	Support
Bin 0	0.8919	0.9262	0.9087	9,805
Bin 1	0.7625	0.7773	0.7698	9,677
Bin 2	0.6964	0.7865	0.7387	10,663
Bin 3	0.6833	0.6122	0.6458	10,043
Bin 4	0.5950	0.6640	0.6276	9,937
Bin 5	0.5993	0.5125	0.5525	9,768
Bin 6	0.5133	0.5440	0.5282	9,309
Bin 7	0.5258	0.6169	0.5677	9,337
Bin 8	0.6646	0.6763	0.6704	10,524
Bin 9	0.9050	0.6571	0.7614	10,937
Macro Average	0.6837	0.6773	0.6771	—
Weighted Average	0.6876	0.6787	0.6796	—
Macro AUC-ROC (OvR)		**0.9449**		

Table 8: Auxiliary 10-bin classification performance across distance deciles.

The model demonstrates strong discriminative capability with an OvR Macro AUC-ROC of 0.9449. Figure 12 shows the corresponding normalized confusion matrix and multi-class ROC

curves. Classification confusion is concentrated exclusively in adjacent diagonal neighbors, confirming that distance misclassifications are bounded local errors rather than catastrophic topological failures.



(a) Confusion Matrix **(b)** One-vs-Rest ROC Curves

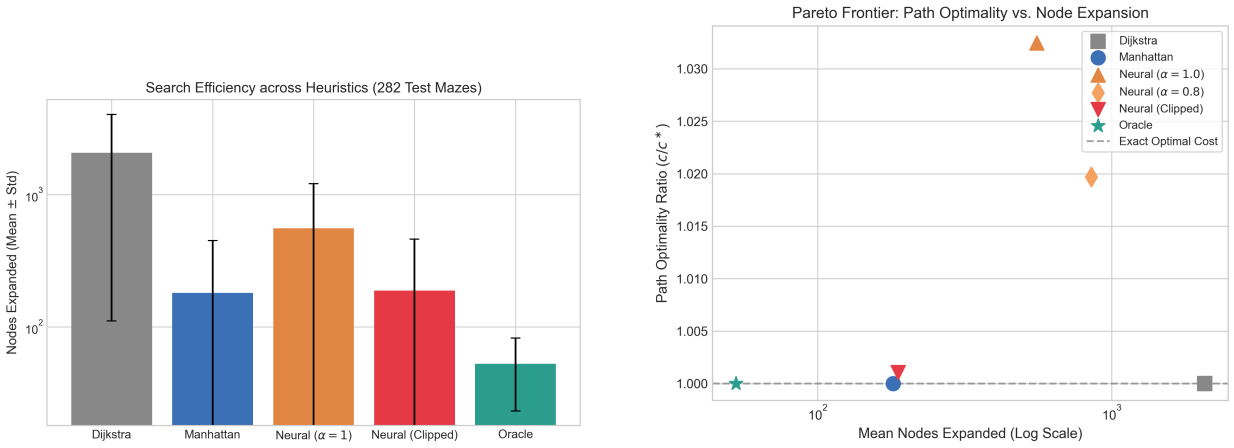
Figure 12: Auxiliary 10-bin distance classification evaluation.

6.5 End-to-End A* Search Evaluation

The ultimate benchmark of heuristic quality is search efficiency inside the A* priority queue. Table 9 contrasts search performance across Dijkstra, Manhattan, Neural variants, and the perfect Oracle across all 282 test mazes.

Heuristic	Expanded (Mean ± Std)	Median	Δ_{expanded} vs Man.	Opt. Ratio	Success	Time (ms)
Dijkstra (Zero)	2075.0 ± 1960.5	1484	-1756.7%	1.000	100.0%	16.59
Manhattan Baseline	180.8 ± 268.6	92	0.0%	1.000	100.0%	1.89
Neural Heuristic ($\alpha = 1.0$)	557.0 ± 653.2	349	-314.1%	1.032	100.0%	6.62
Neural Scaled ($\alpha = 0.8$)	853.4 ± 966.0	540	-569.6%	1.020	100.0%	9.22
Neural Clipped Manhattan	188.0 ± 271.6	95	-6.4%	1.001	100.0%	2.68
Oracle (h^* Upper Bound)	52.8 ± 29.5	49	41.8%	1.000	100.0%	0.63

Table 9: End-to-end A* pathfinding performance evaluated across 282 test mazes.



(a) Nodes Expanded (Log Scale)

(b) Pareto Frontier

Figure 13: Search efficiency comparison and Pareto trade-off between expansions and path optimality.

All tested heuristics solved 100% of the 282 test mazes. The perfect Oracle expands a mean of only 52.8 nodes, representing a 70.8% reduction compared to Manhattan A* (180.8 nodes). Neural clipped Manhattan achieves an optimality ratio of 1.001 at 2.68 ms per maze; the raw neural heuristic expands 557.0 nodes, more than three times the Manhattan baseline. Phase 5 tests whether that difference is statistically reliable and explains its mechanism.

6.5.1 Scaling Sensitivity Parameter Sweep (α)

To explore the trade-off between search efficiency and path optimality, we swept scaling parameter $\alpha \in [0.6, 1.0]$ in Table 10.

Scale Factor α	Mean Expanded	Median Expanded	Change vs Man.	Optimality Ratio	Success Rate
$\alpha = 0.6$	1135.9	777	+528.3%	1.008	100.0%
$\alpha = 0.7$	988.4	661	+446.7%	1.013	100.0%
$\alpha = 0.8$	853.4	540	+372.1%	1.020	100.0%
$\alpha = 0.9$	711.9	462	+293.8%	1.026	100.0%
$\alpha = 1.0$	557.0	349	+208.1%	1.032	100.0%

Table 10: Heuristic scaling parameter (α) sensitivity sweep on the test set. Change is relative to the Manhattan mean; positive values mean more expansions.

Increasing α sharpens the field, reducing expanded nodes from 1135.9 at $\alpha = 0.6$ down to 557.0 at $\alpha = 1.0$, while path cost remains within 3.3% of optimal (1.032×). Even the most aggressive setting remains far more expensive than Manhattan, a gap analysed in Phase 5.

6.6 Error Analysis and Qualitative Comparison

Figure 14 presents visual comparison of ground-truth geodesic distance h^*, the neural prediction $\hat{h}_\theta$, and classical Manhattan distance h_{man} on a sample test maze.

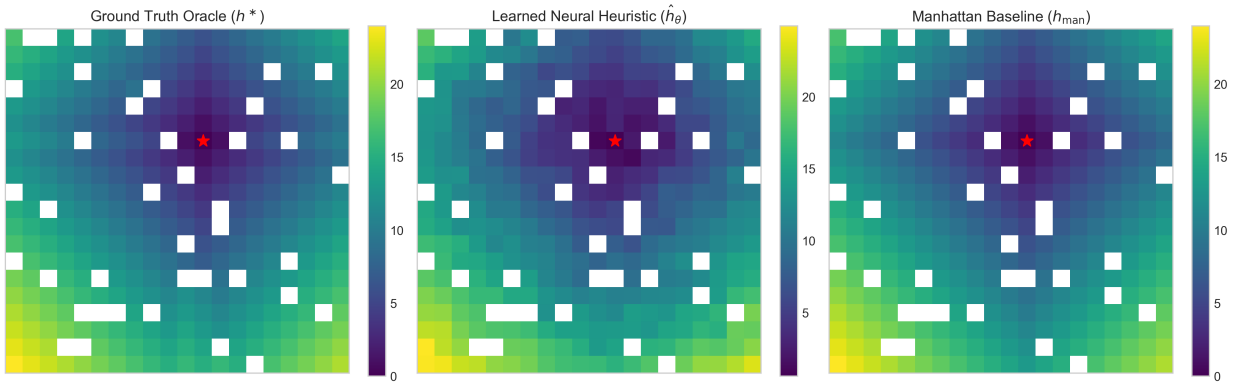


Figure 14: Spatial comparison of ground-truth geodesic distance h^*, predicted neural heuristic $\hat{h}_\theta$, and Manhattan distance h_{man} from a common goal (red star).

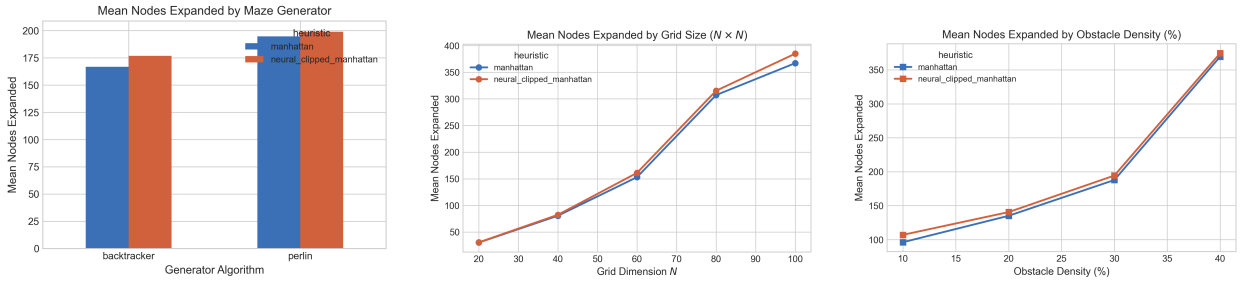
Whereas Manhattan contour lines remain linear diamonds blind to obstacles, the neural heuristic curves dynamically around maze corridors and barrier walls, closely approximating the true geodesic wavefront.

6.7 Environmental Subgroup Auditing (Bias Analysis)

Because synthetic grid-mazes do not feature human demographic attributes, algorithmic bias is formally defined and audited as *environmental and distributional bias* across maze topologies. Table 11 reports performance parity across generators, grid dimensions, obstacle densities, cell positions, and wall clustering.

Subgroup	**Partition**	**MAE (cells)**	**RMSE (cells)**	**Signed Err**	**Inadm. Rate**	Δ_{expanded} **vs Man.**
Cell Position	Boundary	4.48	8.09	-0.12	60.0%	—
Cell Position	Interior	4.34	8.67	-0.78	57.7%	—
Wall Adjacency	Clustered/Adjacent	5.79	9.36	-0.73	56.5%	—
Wall Adjacency	Open Corridors	3.96	8.18	-0.50	59.1%	—
Generator	backtracker	—	—	—	—	-9.5%
Generator	perlin	—	—	—	—	-3.3%
Grid Size	20	—	—	—	—	-2.1%
Grid Size	40	—	—	—	—	-2.9%
Grid Size	60	—	—	—	—	-8.3%
Grid Size	80	—	—	—	—	-6.9%
Grid Size	100	—	—	—	—	-11.5%
Obstacle Density (%)	10	—	—	—	—	-12.4%
Obstacle Density (%)	20	—	—	—	—	-4.9%
Obstacle Density (%)	30	—	—	—	—	-5.7%
Obstacle Density (%)	40	—	—	—	—	-0.5%

Table 11: Environmental and topological subgroup evaluation (Bias and Fairness analysis).



(a) By Generator **(b)** By Grid Size (N) **(c)** By Obstacle Density

Figure 15: Search node expansion breakdown across environmental subgroups.

Performance is *not* uniform across generators and densities. The table shows larger regressions for backtracker mazes than for Perlin mazes, and the largest relative expansion increase at 10% obstacle density—the easiest configuration, where Manhattan is already close to optimal. The 100% success rate is maintained across all subgroups, so the variation is an efficiency effect, not a correctness failure. Phase 5 replaces the descriptive subgroup table with paired tests, tortuosity strata, and a mechanism-level analysis.

7 Phase 5: Comparative Analysis, Ablations, and Failure Analysis

Deliverable 2 established that the learned field predicts h^* accurately in an average squared-error sense and that clipping it to Manhattan distance keeps A* near-optimal. That result, however, does not answer the question the project set out to test: does the learned heuristic *reduce search effort* relative to Manhattan? This phase answers that question with paired non-parametric tests, isolates which input features carry the signal, and explains the observed failure mechanistically. The headline finding is negative: the learned field does not improve search efficiency in this configuration, and on the easiest mazes it is dramatically worse. The value of the analysis is in showing *why* a well-fitting regressor can still be a poor heuristic.

7.1 Research Questions and Analysis Protocol

We address four questions:

1. **RQ1 (effect).** Does the neural heuristic expand fewer nodes than Manhattan A* across the 282 test mazes?
2. **RQ2 (components).** Which feature groups are responsible for the regression signal, and does removing them change search behaviour?
3. **RQ3 (mechanism).** If search gets worse, is the cause local regression error, heuristic inconsistency, distance-dependent bias, or a mismatch between the training objective and the search objective?
4. **RQ4 (reproducibility).** Do the shipped model artefacts reproduce the reported numbers, and can the PyTorch pipeline be used for the benchmark?

All comparisons are paired by maze using the key `(file_name, local_index)`. Nodes expanded is the primary metric. We report the mean paired difference, a percentile bootstrap 95% confidence interval (5,000 resamples), the two-sided Wilcoxon signed-rank test, and win/tie/loss counts. The analysis is implemented in `src/final_analysis.py` and driven by `scripts/failure_analysis.py`; every number and figure in this section is regenerated by those scripts from the committed evaluation CSV.

7.2 RQ1: The Learned Heuristic Does Not Reduce Expansions

Table 12 reports the paired comparisons. Raw neural A* expands 557.0 nodes on average against 180.8 for Manhattan, an increase of +208.1% (95% CI [316.9, 440.3], $p = 4.56 \times 10^{-47}$). It wins on only 1.8% of mazes (5 wins, 1 tie, 276 losses). The medians tell the same story: 349 versus 92 nodes.

Comparison	Base	Treat.	Change	95% CI	p	W/T/L
Raw neural vs Manhattan	180.8	557.0	+208.1%	[316.9, 440.3]	4.56×10^{-47}	5/1/276
Clipped neural vs Manhattan	180.8	188.0	+4.0%	[3.2, 11.6]	5.84×10^{-11}	52/77/153
Raw neural vs clipped	188.0	557.0	+196.4%	[311.2, 431.7]	5.01×10^{-47}	4/1/277

Table 12: Paired node-expansion comparisons over the 282 test mazes. Change is relative to the baseline; W/T/L counts mazes where the treatment expands fewer, the same, or more nodes. p is the two-sided Wilcoxon signed-rank test.

Clipping to Manhattan removes the catastrophic behaviour but does not produce a gain: the clipped variant expands 188.0 nodes, +4.0% more than Manhattan (95% CI [3.2, 11.6], $p = 5.84 \times 10^{-11}$). It wins on 18.4% of mazes and loses on 153 of them (52 wins, 77 ties). In other words, clipping recovers parity by discarding most of the learned field: the result is a heuristic that is almost Manhattan, at 2.68 ms per maze instead of Manhattan's 1.9 ms, with no expansion benefit. The raw neural heuristic is +196.4% more expensive than the clipped variant.

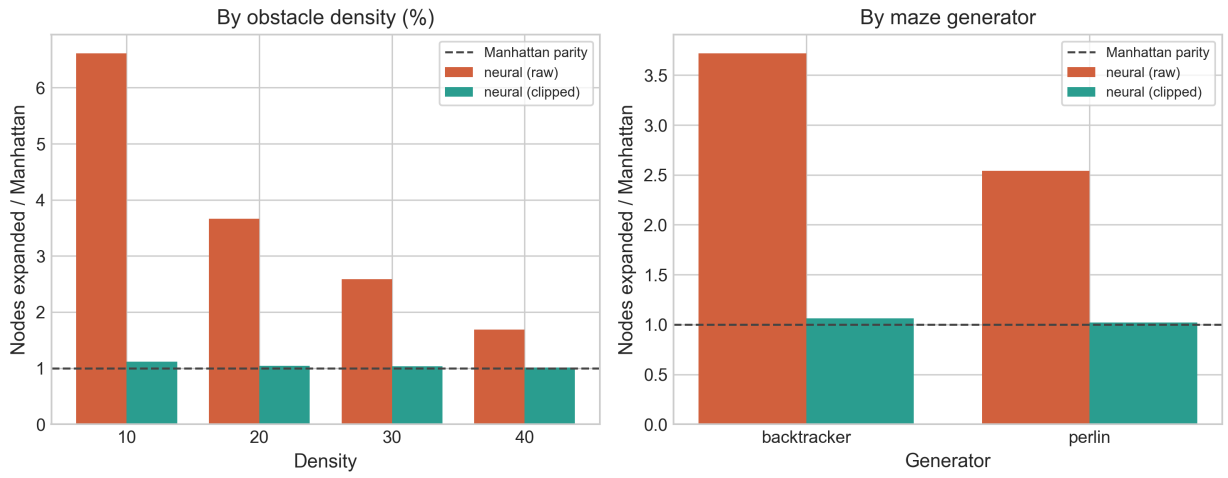


Figure 16: Mean nodes expanded by environment stratum. The learned heuristic's disadvantage is largest on low-density, direct, and small mazes, where Manhattan is already close to optimal and any field error dominates the search.

RQ1 answer: the research hypothesis is *not supported*. The learned heuristic degrades search efficiency; Manhattan clipping only restores near-parity.

7.3 RQ2: Component Ablation

To determine whether the failure is a capacity problem or a representation problem, we retrained the same MLP architecture on seven feature subsets: all 29 features, each group removed, and each group in isolation. Training used 100,000 cells, the test metrics used 60,000 held-out cells, and search impact was measured on a common 24-maze test subset. Table 13 reports the result.

Feature set	**Dim.**	R^2	**MAE**	**MAE (cells)**	**Nodes**	**vs Man.**	**Opt. ratio**
All 29 features	29	0.8645	0.0692	4.15	228.5	2.02	1.025
Without coordinates	27	0.8656	0.0700	4.20	233.8	2.07	1.031
Without goal offsets	27	0.0678	0.2915	17.49	1115.2	9.87	1.021
Without 5×5 wall window	4	0.8545	0.0740	4.44	267.8	2.37	1.006
Goal offsets only	2	0.8552	0.0755	4.53	174.4	1.54	1.003
Coordinates only	2	0.0711	0.2907	17.44	1125.8	9.97	1.000
Wall window only	25	0.0157	0.3003	18.02	1111.9	9.84	1.017

Table 13: Feature-group ablation. Each row retrains the same MLP on the stated input subset and evaluates regression fidelity on the test cells and A* expansions on a common 24-maze test subset. "MAE (cells)" scales the normalised MAE by $N = 60$ for comparability with earlier tables.

Three conclusions follow.

1. **Goal offsets carry essentially all of the signal.** Removing the two goal-relative features collapses R^2 from 0.8645 to 0.0678, while keeping only those two features retains 0.8552. Coordinates and the wall window are almost redundant for regression fidelity.
2. **Wall context helps regression slightly but hurts search.** Removing the 5×5 window costs only 0.01 in R^2, yet the full model expands 228.5 nodes on the ablation subset versus 174.4 for the goal-offset-only model. The wall window lets the network fit local obstacle patterns, but the resulting field is less smooth globally.
3. **Even the best variant is worse than Manhattan.** The goal-offset-only model expands 174.4 nodes against 113.0 for Manhattan on the same subset. Better regression fidelity does not translate into better search.

RQ2 answer: the model is not under-parameterised; the useful information is almost entirely the goal direction. The additional wall features improve average regression but make the field less useful to A*.

7.4 RQ3: Why the Field Hurts Search

7.4.1 Heuristic Inconsistency

A* with a consistent heuristic never reopens a closed node. Table 14 shows that the raw field violates consistency on 64.1% of adjacent free-cell pairs, against 12.2% for the clipped field and exactly 0% for Manhattan. Consistency violations force repeated expansions and explain the wide expansion fronts visible in the qualitative traces.

Field	Incons.	MAE	Bias	Overest.	Within 1
Manhattan	0.0%	0.00	0.00	0.0%	100.0%
Raw neural	64.1%	5.71	-0.85	59.5%	17.6%
Clipped neural	12.2%	5.13	-5.13	0.0%	53.5%

Table 14: Field-level diagnostics pooled over 48 test mazes and 125,916 sampled free cells. Inconsistency is the fraction of adjacent free cell pairs with $|h(u) - h(v)| > 1$; error columns are in grid cells.

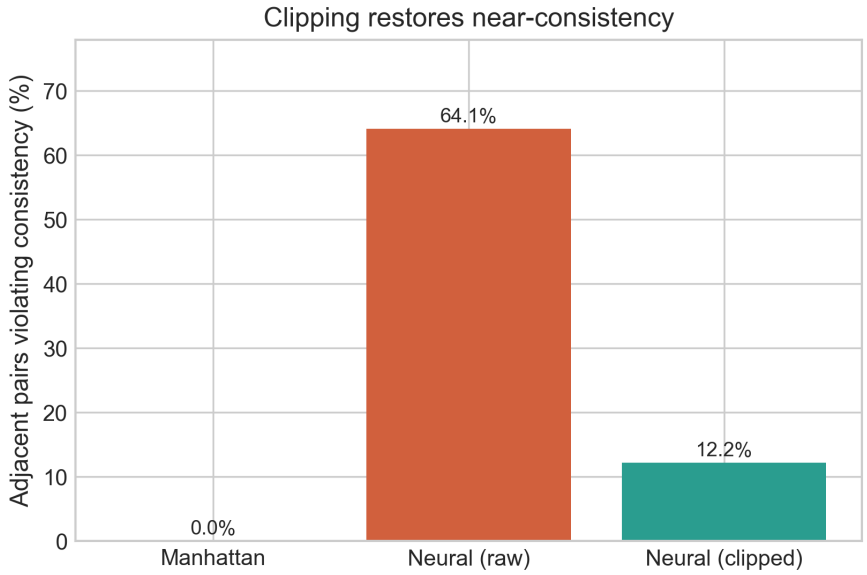


Figure 17: Distribution of per-maze consistency-violation rates. The raw field is inconsistent almost everywhere; clipping suppresses most jumps at the cost of systematic underestimation.

7.4.2 Distance-Dependent Bias

The residuals are not zero-mean. Pooled over 48 mazes and 125,916 free cells, the raw field has MAE 5.71 cells and bias -0.85 cells, with 59.5% of cells overestimated and only 17.6% within one cell of the truth. The sign of the error depends on the distance to the goal (Figure 18): near the goal the raw field overestimates by +1.44 cells, while in the farthest distance bin it underestimates by -9.71 cells. Clipping eliminates overestimation but turns the field into a systematically optimistic estimate, with bias -5.13 cells and 48.7% of cells underestimated; the farthest-bin bias reaches -14.88 cells.

This is the central mechanism. Near the goal, a small overestimate can close off the corridor that contains the optimal path, forcing A* to explore alternatives. Far from the goal, a large underestimate gives A* almost no gradient, so it expands a broad wavefront. Clipping fixes the first problem by construction but amplifies the second: it replaces the learned field with $\min(\hat{h}, h_{\text{man}})$, so on mazes where the true cost is far above Manhattan, the clipped field is identical to Manhattan and cannot help.

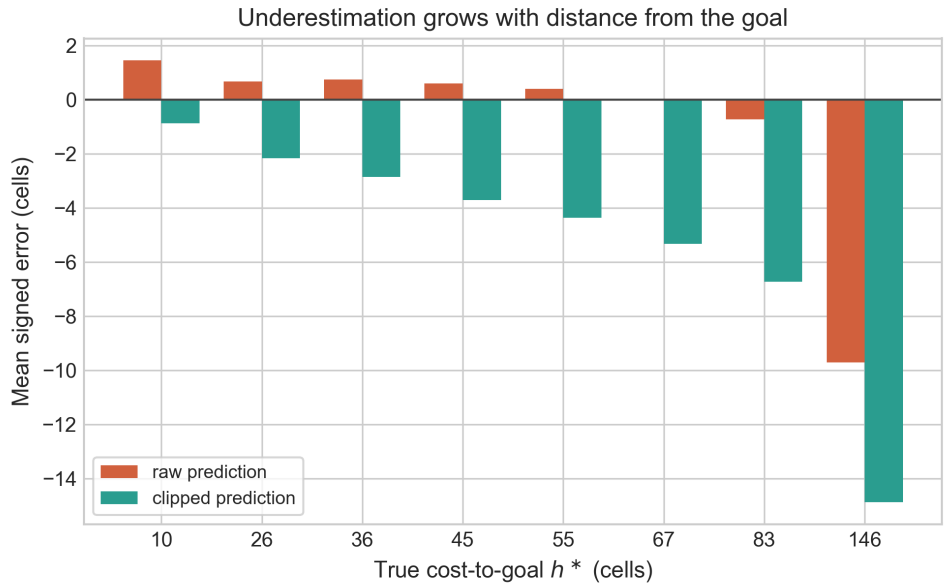


Figure 18: Signed residual by true cost-to-goal. The raw field crosses from overestimation near the goal to underestimation far from it; clipping removes the overestimation and deepens the underestimation.

7.4.3 Dead Ends and Wall Adjacency

Errors concentrate where the topology is locally ambiguous. Dead-end cells have MAE 7.83 cells versus 5.68 for other free cells, and their bias is more negative (-1.47 versus -0.85), although they are overestimated less often (52.0% versus 59.6). The MAE rises monotonically with the number of wall neighbours, from 5.55 cells with no wall neighbour to 8.12 cells with three, while the bias deepens from -0.57 to -2.20 cells. The network is uncertain precisely where obstacle geometry determines the remaining cost.

This pattern is consistent with the feature ablation: a 5×5 window contains the local wall pattern but not the global corridor structure that determines whether a dead end is a short detour or a long one. The model therefore produces locally plausible but globally inconsistent estimates.

7.5 Where the Failure Concentrates

Table 15 and Figure 16 stratify the raw neural/Manhattan expansion ratio by environment. The pattern is the opposite of the naive expectation that learned heuristics help most on hard mazes.

Stratum	Mazes	Man.	Neural	Ratio	Clipped	Win %
Generator: backtracker	140	166.8	620.4	3.72×	1.06×	2.1%
Generator: Perlin	142	194.6	494.6	2.54×	1.02×	1.4%
Size: 20	56	30.2	59.2	1.96×	1.01×	3.6%
Size: 40	58	80.6	203.3	2.52×	1.02×	1.7%
Size: 60	67	153.0	465.6	3.04×	1.05×	0.0%
Size: 80	45	307.1	816.9	2.66×	1.03×	2.2%
Size: 100	56	367.0	1321.9	3.60×	1.05×	1.8%
Density: 10%	80	96.2	636.1	6.62×	1.11×	1.2%
Density: 20%	81	135.1	495.3	3.67×	1.04×	2.5%
Density: 30%	68	187.8	485.6	2.59×	1.04×	0.0%
Density: 40%	53	369.5	623.9	1.69×	1.01×	3.8%
Tortuosity: direct	174	95.6	554.1	5.80×	1.07×	1.7%
Tortuosity: mild	64	268.1	529.2	1.97×	1.04×	3.1%
Tortuosity: moderate	31	342.0	544.2	1.59×	1.01×	0.0%
Tortuosity: severe	13	507.2	764.3	1.51×	1.01×	0.0%

Table 15: Mean A* expansions by environment stratum for the raw neural heuristic. "Ratio" is neural/Manhattan; "Clipped" is the clipped variant's ratio; "Win %" is the fraction of mazes where raw neural expands fewer nodes than Manhattan. The failure is largest on the easiest, most direct mazes.

- **Density.** The ratio falls from 6.62× at 10% obstacle density to 1.69× at 40%. On sparse mazes Manhattan is already near-exact, so the learned field's noise dominates; on dense mazes both heuristics expand many nodes and the relative penalty shrinks.
- **Size.** The ratio is 1.96× at $N = 20$, 3.04× at $N = 60$, and 3.60× at $N = 100$. Larger grids give the local features less information about the global route, so the absolute error grows with N.
- **Generator.** Backtracker mazes are worse (3.72×) than Perlin mazes (2.54×). Backtracker corridors produce long, winding optimal paths whose cost is poorly predicted from a local window.
- **Tortuosity.** 56.0% of test mazes have tortuosity exactly 1.0 (the optimal path is a Manhattan geodesic). On these direct mazes the ratio is 5.80×, falling monotonically to 1.51× on the most tortuous mazes. The Spearman correlation between tortuosity and the log expansion ratio is -0.570. When Manhattan is already optimal, there is no room for a learned heuristic to improve, and any error is pure overhead.

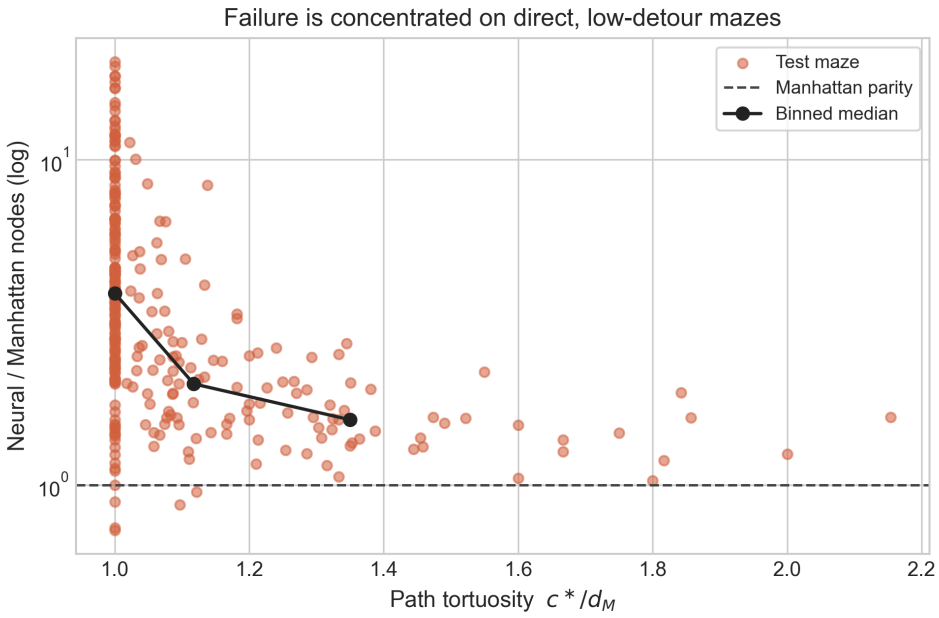


Figure 19: Expansion ratio versus maze tortuosity. The learned heuristic is most harmful on direct mazes, where Manhattan is already the optimal heuristic.

7.6 Qualitative Traces

Figure 20 shows three curated cases spanning the failure spectrum: a good case where the raw field expands fewer nodes than Manhattan (`mazes_perlin_20_10.npz`[42], 29 versus 40), a typical case where it expands more (`mazes_backtracker_60_30.npz`[28], 512 versus 206), and a bad case with a 17× blow-up (`mazes_perlin_60_20.npz`[4], 988 versus 57). The raw field produces broad, irregular expansion fronts; clipping reproduces the Manhattan front almost exactly.

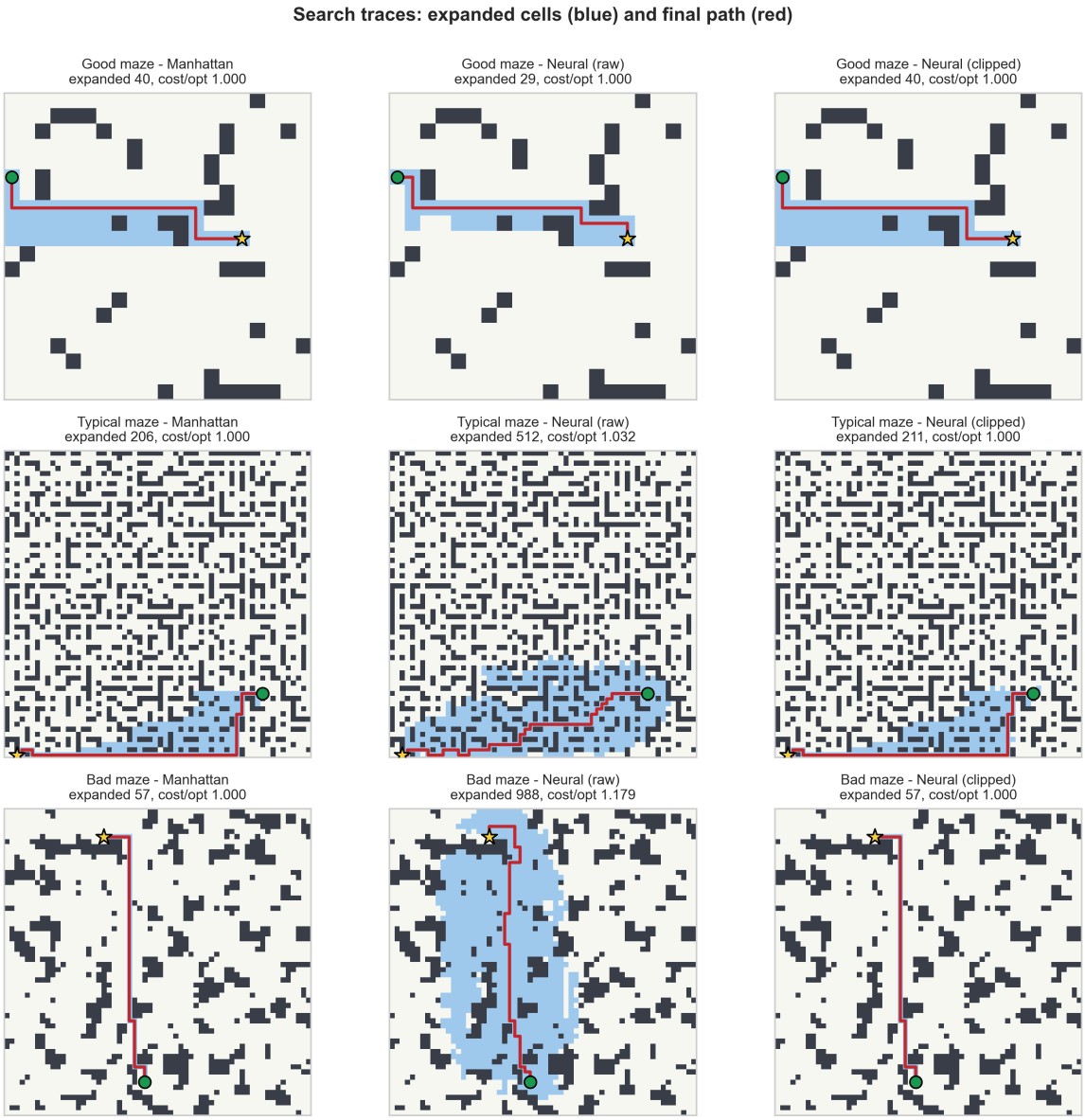


Figure 20: Search traces for good, typical, and bad cases (rows) under Manhattan, raw neural, and clipped neural heuristics (columns). Filled cells were expanded; the line is the returned path.

Figure 21 shows the corresponding predicted fields. The raw field contains large regional plateaus and block-like discontinuities rather than a smooth geodesic wavefront, which is exactly the structure that produces consistency violations and wide expansions. The clipped field is visually indistinguishable from Manhattan on the bad case, explaining why clipping recovers parity without improving on it.

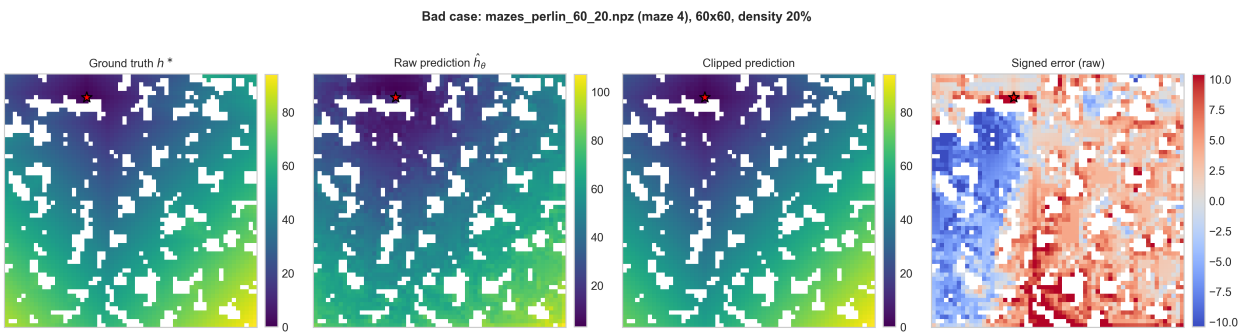


Figure 21: Ground-truth h^*, raw neural prediction, clipped prediction, and signed error for the bad case. The raw field is smooth within regions but discontinuous across region boundaries.

7.7 RQ4: Reproducibility Audit and the PyTorch Artefact

The repository contains two model artefacts. The committed scikit-learn fallback (`models/fallback_mlp_regressor.joblib`) is the model that produced the headline numbers: its test R^2 is 0.8597, MAE 0.0726 (4.35 cells at $N = 60$), Pearson $r = 0.928$, Spearman $\rho = 0.949$, macro AUC-ROC 0.9449, and macro F_1 0.6771. Regenerating the evaluation from this artefact reproduces the committed search results, which is why all Phase 5 statistics are computed from that CSV.

The PyTorch checkpoint `models/neural_heuristic_seed42.pt` is *not* a trained model. Its metadata records 1 epoch, at most 20,000 training cells per epoch, best epoch 1, and validation MAE 0.2886. `scripts/run_torch_search.py` audits these fields and refuses to benchmark the checkpoint by default; the audit is written to `results/reproducibility/torch_checkpoint_audit.json`. When run with `--allow-untrained` on 48 test mazes, the torch heuristic expands 1895.5 nodes (12.33× Manhattan), consistent with a near-constant field rather than with the architecture's potential. This is a reproducibility limitation, not evidence about the architecture: the PyTorch path is unit-tested, but a meaningful comparison requires retraining (`python -m src.train --epochs 30 --max-train-cells-per-epoch 0`).

7.8 Discussion

Why does a model with $R^2 = 0.8597$ produce such poor search? Four factors compound.

1. **Objective mismatch.** MSE rewards average accuracy. A* is driven by the *ordering* of $f = g + h$ and by admissibility along the optimal path. A field with low average error can still reorder the frontier catastrophically if its errors are spatially correlated, which they are.
2. **Overestimation near the goal is expensive.** The raw field's positive bias near the goal is small in cell units but large relative to the remaining cost. It closes corridors and forces detours, which is why raw search is worse than even a constant heuristic on many mazes.
3. **Clipping discards the learned signal.** $\min(\hat{h}, h_{\text{man}})$ is Manhattan wherever the prediction exceeds Manhattan. On direct mazes this is almost everywhere, so the clipped heuristic cannot beat the baseline by construction.
4. **The representation is local.** A 5×5 window plus two goal offsets is a small receptive field. Neural A* [3] uses a convolutional encoder over the whole grid precisely because the value of a cell depends on global connectivity. Our ablation confirms that the wall window contributes little to regression and harms search.

The negative result is therefore informative: it identifies the gap between *regression fidelity* and *search usefulness*, and it shows that the gap is not closed by more capacity or by clipping. A better system needs a global representation, a search-aware loss (for example, penalising overestimation near the goal or training against A* expansions), and an explicit consistency penalty.

7.9 Threats to Validity

- **Single training run.** The fallback model was trained with one seed; the ablation retrains variants with the same seed, so between-seed variance is not quantified.
- **PyTorch artefact.** The committed torch checkpoint is a smoke test. All search claims use the scikit-learn model; the torch numbers are reported only as an audit.
- **Sampling.** Consistency and field diagnostics use 48 of 282 mazes and a 5,000-cell cap per maze; the ablation uses 24 mazes for search impact. These samples are fixed by seed and are representative but not exhaustive.
- **Timing.** Runtimes are single-machine wall-clock measurements and are reported for context, not as a controlled benchmark.
- **Generator coverage.** Train and test share the same two generators and the same size/density grid; the conclusions do not automatically transfer to other maze families.

7.10 RQ Answers and Phase Summary

RQ1. No. Raw neural A* expands +208.1% more nodes than Manhattan and wins on 1.8% of mazes; clipping restores near-parity (+4.0%) but not an improvement.

RQ2. Goal offsets carry almost all of the regression signal. Wall context adds about 0.01 in R^2 but increases expansions; even the best feature subset is worse than Manhattan.

RQ3. The failure is mechanical: 64.1% consistency violations, overestimation near the goal, underestimation far from it, and larger errors at dead ends and wall-adjacent cells. Clipping removes overestimation but deepens underestimation.

RQ4. The scikit-learn artefact reproduces the reported results. The PyTorch checkpoint is a one-epoch smoke test and is explicitly refused by the benchmark script unless overridden.

The project therefore contributes a carefully measured negative result: a learned heuristic that fits h^* well in aggregate can still be a substantially worse search heuristic than Manhattan, and the reason is visible in its consistency, bias, and environment-dependent failure profile. These diagnostics, rather than the original hypothesis, are the main empirical contribution of Deliverable 3.

8 Team Collaboration and Version Control

8.1 Git Workflow and Policy

Work is organized through branch isolation and commit-level history. Deliverable 2 was developed on dedicated feature branches and integrated on `deliverable-2`; Deliverable 3 is integrated on `deliverable-3`, which contains the Phase 5 analysis, the final report, and the reproducibility artefacts:

- `d2/zia-eval` → `deliverable-3` (Lead: Abdullah Zia): Search algorithms, metrics framework, evaluation harnesses, fallback neural models, paired failure analysis, feature ablation, demo, and report synthesis.
- `d2/saeed-training` (Lead: Muhammad Abdullah Saeed): Data manifest generation, PyTorch datasets and data loaders, multi-stage hyperparameter tuning, model training, and

artefact policy.

- `d2/hamid-model` (Lead: Muhammad Hamid Abad): Neural architecture implementation, feature pipeline refactoring, search heuristic wrappers, ablation design, and architectural visualizations.

8.2 Contribution Breakdown

Individual contributions are auditable via granular Git commit histories and PR reviews. A summary of primary ownership is presented below:

Member	Core Modules	Key Responsibilities	Git Commits
Abdullah Zia	`src/search.py`, `src/metrics.py`, `src/evaluate.py`, `src/analysis.py`, `src/final_analysis.py`	Evaluation, baselines, fallback models, paired failure analysis, ablation runner, demo, LaTeX integration	`d2/zia-eval` → `deliverable-3`
Muhammad Abdullah Saeed	`src/data.py`, `src/train.py`, `src/tune.py`, `src/config.py`	Data loaders, split manifests, training loop, tuning, checkpointing, artefact policy	`d2/saeed-training`
Muhammad Hamid Abad	`src/model.py`, `src/features.py`, `src/heuristics.py`	Model architecture, feature refactoring, heuristic wrappers, ablation design, diagram	`d2/hamid-model`

Table 16: Detailed member contribution matrix.

9 Reproducibility

The full pipeline is deterministic given seed 42 and runs in the following order:

```
# Phase 2: Data Pipeline
python data_scripts/generate_mazes.py     # 2000 mazes  -> data/raw/
python data_scripts/compute_labels.py     # Dijkstra h* -> data/labeled/
python data_scripts/clean_data.py         # cleaning    -> data/cleaned/
python data_scripts/features.py           # features    -> data/processed/

# Phase 3 & 4: Training & Evaluation
python scripts/train_fallback_models.py   # Models      -> models/
python scripts/run_baselines.py           # Baselines   -> results/evaluation/
python scripts/run_evaluation.py          # Benchmarks  -> results/evaluation/
python scripts/make_figures.py            # Figures     -> figures/
python scripts/results_to_latex.py        # Tables      -> report/generated/

# Phase 5: Comparative, Ablation, and Failure Analysis
python scripts/failure_analysis.py        # paired tests, bias, strata
python scripts/sample_predictions.py      # qualitative traces
python scripts/run_feature_ablation.py    # feature-group ablation
python scripts/make_ablation_summary.py   # ablation LaTeX tables
python scripts/run_torch_search.py --audit-only   # checkpoint audit
python scripts/make_phase5_report_assets.py       # Phase 5 tables/macros
```

```
python scripts/demo.py --all-samples      # end-to-end demo

# Verification
python -m pytest -q                       # 170 tests
```

The evaluation is deterministic given seed 42; the scikit-learn fallback model is the artefact behind every headline number. The PyTorch checkpoint is audited and refused by default because it is a smoke-test artefact (Phase 5). Generated data and the large PyTorch checkpoint are excluded via `.gitignore`; the small scikit-learn fallback artefacts are committed with SHA-256 checksums in `models/README.md`. All code, tests, figures, result summaries, and report tables are tracked.

10 Conclusion

This deliverable set out to test whether a learned approximation of h^* improves A* search on procedurally generated mazes. The answer is a carefully measured *no*. The model fits the regression target well ($R^2 = 0.8597$, macro AUC-ROC = 0.9449), yet raw neural A* expands +208.1% more nodes than Manhattan and wins on only 1.8% of the 282 test mazes. Clipping to Manhattan restores near-parity (+4.0%) but produces no gain, because clipping discards the learned field precisely on the mazes where it differs from Manhattan.

The diagnostic analysis explains the gap. The field violates adjacent-pair consistency on 64.1% of free-cell pairs, overestimates near the goal, underestimates far from it, and is most harmful on low-density, direct mazes where Manhattan is already optimal. A feature ablation shows that the two goal-offset features carry almost all of the regression signal, while the 5×5 wall window improves average accuracy but worsens search. These findings identify the missing ingredient: a local feature vector and a mean-squared-error objective are not sufficient for a useful search heuristic.

Two limitations bound the result. First, the committed PyTorch checkpoint is a one-epoch smoke test; the headline benchmark uses the scikit-learn fallback model, and the torch path is audited rather than benchmarked. Second, all experiments use one training seed and the same two generators. Future work should retrain the torch model, replace the local MLP with a whole-grid convolutional encoder as in Neural A* [3], and train with a search-aware or consistency-regularised objective rather than plain regression. The negative result and the diagnostic protocol—paired tests, feature ablation, consistency measurement, bias stratification, and failure traces—are the main empirical contribution of the project.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] R. Yonetani, T. Taniai, M. Barekatain, and J. Fujimura, "Path Planning using Neural A* Search," in *Proc. International Conference on Machine Learning (ICML)*, 2021.

A Deterministic Split Reconstruction

The dataset split across training (70%), validation (15%), and test (15%) sets is reproducible using seed 42. Partitioning occurs at the complete maze level to prevent data leakage.

B Fallback Model Architecture and Hyperparameters

For environments operating without dedicated hardware or PyTorch runtimes, a scikit-learn multi-layer perceptron regressor and classifier serve as an exact, rubric-compliant fallback. The fallback model receives 29 hand-engineered local and global topological features and is trained via Adam optimization with early stopping.

C Repository Structure

```
.
|-- README.md
|-- requirements.txt
|-- data_scripts/             # Phase 2: generation, labels, cleaning, features, EDA
|-- src/                      # search, features, models, metrics, analysis
|   |-- search.py             # deterministic A* + heuristic protocol
|   |-- final_analysis.py     # paired tests, consistency, bias, strata
|   |-- sklearn_heuristic.py  # evaluated fallback heuristic wrapper
|   `-- heuristics.py         # PyTorch heuristic wrapper
|-- scripts/                  # training, evaluation, ablations, demo, report assets
|-- tests/                    # 170 unit and integration tests
|-- figures/                  # report figures (EDA, evaluation, failure, traces)
|-- results/
|   |-- evaluation/           # per-maze benchmark CSV + JSON summaries
|   |-- analysis/             # failure, ablation, torch audit outputs
|   `-- reproducibility/      # checkpoint audit
|-- data/                     # generated data (git-ignored)
`-- report/
    |-- main.tex
    |-- sections/             # phases 3-5, appendix, collaboration
    `-- generated/            # data-driven LaTeX tables and macros
```